API Documentation

Ta
of
Co

App-specific OpenAPI reference

ACE eCom Feed Test

ACE Social Media Video (Seb)

The Brainsuite API lets you start jobs and receive reports for your apps.

Welcome to the Brainsuite documentation. Follow the instructions below to learn how to integrate the Brainsuite API.

Authentication

Brainsuite uses API Clients to authenticate and authorize programmatic access. Each API Client represents an application or integration, and is managed within the Brainsuite Platform. Brainsuite's implementation is based on the Client Credentials Grant flow of the OAuth 2 specification.

Manage API Clients

All API Clients are listed under Administration → API → API Clients.

From this page you can:

- **View** an API Client to access its details
- **Edit** an API Client to update its name or permissions (Admin role required)
- **Delete** clients that are no longer in use. (Admin role required)

If no API Client is listed, contact your account manager or submit a support ticket to request a new API Client.

Obtain client credentials

When you view an API Client, you'll see the credentials which include:

- `Client ID`
- `Client Secret`

> We recommend storing the client credentials as a secret and using a solution like dotenv to load it into your environment. This will prevent you from exposing it as plain text.

Set permissions

Each API Client is assigned permissions that define what parts of Brainsuite it can access. Permissions are managed per client in the permissions table when viewing an API Client.

The following permission categories are available:

- **Reports – View Only**
  Allows the client to read reports but not create new ones.
- **Reports – View + Create**
  Grants both read access and the ability to create new reports.
- **Cockpit**
  Provides access to the Cockpit data of the Brainsuite.

Permissions are applied per app (row in the table), so that you have more granular access levels.

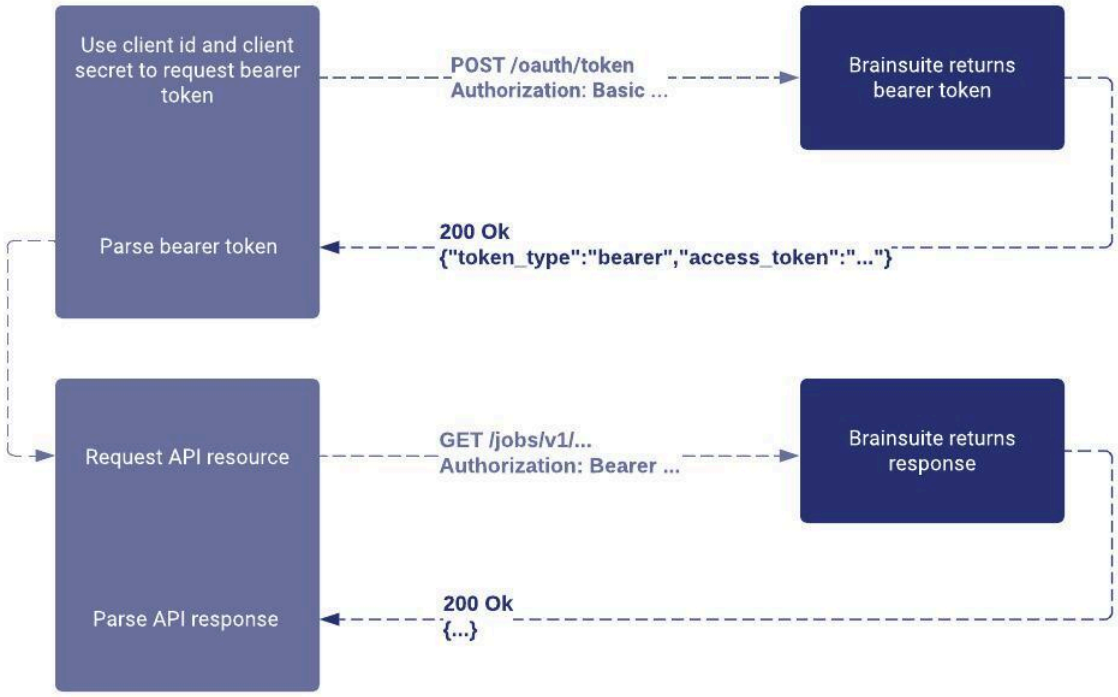
Admin role required for changing permissions.

Auth Flow

To use the API, you need to use an Access Token (also known as Bearer Token). You can generate an Access Token (Bearer Token) by passing your client id and client secret through the `POST oauth2/token` endpoint.

The auth flow follows these steps:

- The customer client encodes its client id and client secret into a specially encoded set of credentials.
- The customer client makes a request to the `POST oauth2/token` endpoint to exchange these credentials for an Access Token.
- When accessing the API, the customer client uses the Access Token to authenticate.



Authentication Steps

Encode client id and secret

The steps to encode a customer client's client id and client secret into a set of credentials to obtain a Bearer Token are:

1. Concatenate the client key, a colon character ":", and the client secret into a single string.

```
client_id:client_secret
```

Copy

2. Base64 encode the string from the previous step.

```
Y2xpZW50X2lkOmNsaWVudF9zZWNyZXQ=
```

Copy

Obtain OAuth 2.0 Bearer Token

You'll need to create an access token, which can be used to make API requests. Successful responses include a JSON-structure describing the awarded Bearer Token.

The value calculated in step 1 must be exchanged for an Access Token by issuing a request to `POST oauth2/token`:

- The request must be an HTTP POST request.
- The request must include an `Authorization` header with the value of Basic `<base64 encoded value from step 1>`.
- The request must include a `Content-Type` header with the value of `application/x-www-form-urlencoded`.
- The body of the request must be `grant_type=client_credentials`.

Resource URL

```
https://auth.brainsuite.ai/oauth2/token
```

Copy

Resource Information

Response formats	JSON
Requires authentication?	Yes - Basic auth with your client id as your username and client secret as your password

Parameters

Name	Required	Description	Example
`grant_type`	required	Specifies the type of grant being requested by the customer client. At this time, only client_credentials is allowed.	*client_credentials*

Example Request

```
POST /oauth2/token HTTP/2
Host: auth.brainsuite.ai
user-agent: insomnia/9.0.0
content-type: application/x-www-form-urlencoded
accept: application/x-www-form-urlencoded, application/json
authorization: Basic eHZ6MWV2R ... o4OERSZHlPZw==
content-length: 29

grant_type=client_credentials
```

Copy

Example Response

```
HTTP/2 200
content-type: application/json;charset=UTF-8
...
```

Copy

```
server: Server

{"token_type":"Bearer", "access_token": "eyJraWdfaews...JSOVZhanhFS", "expire
```

Example Python Code

The customer client should verify that the value associated with the `token_type` key of the returned object is `bearer`. The value associated with the `access_token` key is the Access Token (Bearer Token).

> Only one bearer token may exist outstanding for a customer client, and repeated requests to this method will yield the same already-existent token until it has been invalidated.

Authenticate API requests

The Access Token (Bearer Token) may be used to issue requests to API endpoints. To use the Access Token, construct a normal HTTPS request and include an Authorization header with the value of Bearer `<base64 bearer token value from step 2>`.

Example request (Authorization header has been truncated)

Copy

```
GET /v1/job/567ddb13-fb18-4487-befd-ed4a1e5754c7 HTTP/2
Host: api.brainsuite.ai
user-agent: insomnia/9.0.0
authorization: Bearer eyJraWdfaews...JSOVZhanhFS
accept: */*
```

Rate Limiting

Brainsuite limits the number of REST API requests that you can make within a specific amount of time. This limit helps prevent abuse and denial-of-service attacks, and ensures that the API remains available for all users.

The `POST` endpoints of each app to create or start a job have a primary rate limit. This means that there is a limited amount of jobs you can start via API per app. For more information about your current primary rate limit click here. For all other endpoints a secondary rate limit is applied to ensure the quality of service for all users.

Rate Limiting details

To see your current usage, we provide the following response headers in every `POST` API response:

- `x-ratelimit-limit`: The maximum number of requests you can make in a certain rate limit window.
- `x-ratelimit-used`: The number of requests you have made in the current rate limit window.
- `x-ratelimit-reset`: The time at which the current rate limit window resets, ISO 8601 UTC timestamp.
- `x-ratelimit-resource`: The rate limit resource that the request counted against. Each application has its own rate limits. For more information contact your account manager.

Exceeding the rate limit

If you exceed your primary rate limit, you will receive a 429 response, and the `x-ratelimit-used` header will match the `x-ratelimit-limit` header. You should not retry your request until after the time specified by the `x-ratelimit-reset` header.

If your use case requires a higher limit, please contact us and we'll be happy to discuss your needs.

Start a job

A job generates a report of KPIs that assess the effectiveness of an asset. To start a job, the asset and additional metadata must be submitted.

There are two ways to submit assets when starting a job:

- **Create-Job Workflow**: Use this workflow when assets are available via **public URLs**.
- **Announce-Job Workflow**: Use this workflow when assets are provided as **files** that need to be uploaded.

Each workflow follows a structured process to ensure proper asset handling and job execution. Refer to the sections below for details on each approach.

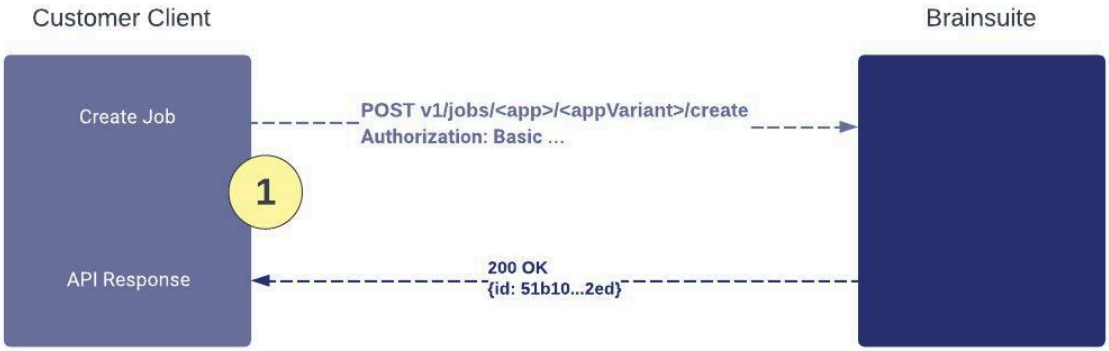
Create-Job Workflow

The Create-Job Workflow is used when assets are available via **public URLs**, allowing them to be referenced directly without the need for manual uploads. This workflow provides a streamlined way to start a job when assets are already available externally.

If assets are not available via public URLs and need to be uploaded manually, refer to the Announce-Job Workflow.

A detailed description of the endpoints specific to the apps are listed in the reference menu on the right or click here.

1. Customer client sends an API request to create a job. The request contains a reference to the assets and additional metadata to create the report. The Brainsuite API endpoint responds with a job ID and the status `Scheduled`.



After a successful creation of the job, use the asynchronous poll-based API to check the status of the started job and to retrieve the report.

Status Information

Announced	Job is announced
Scheduled	Job is scheduled to be processed

Created	Job is processing
Succeeded	Job finished successfully
Failed	Job finished with an error
Stale	Job was not started

Reference multiple assets

Some apps support **multiple assets**, such as a static image and an environment image, allowing different types of assets to be included in a job. Additionally, some apps support **multiple legs**, where each leg represents a variation of an asset.

When using the Create-Job Workflow, assets are provided as public URLs and must be referenced in the request body. Depending on the app's capabilities, both **multiple assets** and **multiple legs** can be configured.

Define assets

Each asset must be included in the assets array, where it is identified by a unique `assetId`.

Copy

```
{
  "assets": [
    {
      "assetId": "staticImage",
      "name": "image.png",
      "url": "https://example.com/path-to-publicly-available-asset"
    },
    {
      "assetId": "environmentImage",
      "name": "background.jpg",
      "url": "https://example.com/path-to-background-image"
    }
  ]
}
```

Note:

- The assets array is mandatory—each job must include at least one asset.
- The `url` field must point to a publicly accessible asset.
- The `assetId` must be unique within the job and is used for referencing assets.

Assign assets to legs (if supported)

If the app supports **multiple legs**, the request input must include a **legs** array. Each leg references an `assetId` from the assets array, ensuring the correct asset is assigned to each variation.

Copy

```
{
  "input": {
    "legs": [
      {
        "name": "Variation 1",
        "staticImage": {
```

```
            "assetId": "staticImage"
          },
          "environmentImage": {
            "assetId": "environmentImage"
          }
        },
        {
          "name": "Variation 2",
          "staticImage": {
            "assetId": "staticImage2"
          },
          "environmentImage": {
            "assetId": "environmentImage"
          }
        }
      ]
    }
  }
```

Note:

- The **legs** array depends on the app's capabilities.
- Each leg must reference an `assetId` that exists in the **assets** array.
- If an app supports different asset types (e.g., `staticImage`, `environmentImage`), they should be linked accordingly.
- A job can contain a maximum number of 10 legs

Summary

- Apps may support **multiple assets**, such as `staticImage` and `environmentImage`.
- Some apps allow **multiple legs**, where each leg represents a different asset variation.
- The **assets** array is **required**, and all assets must be referenced using unique `assetIds`.
- The **legs** array is optional and depends on whether the app supports variations.

For details on app-specific configurations, refer to the OpenAPI reference.

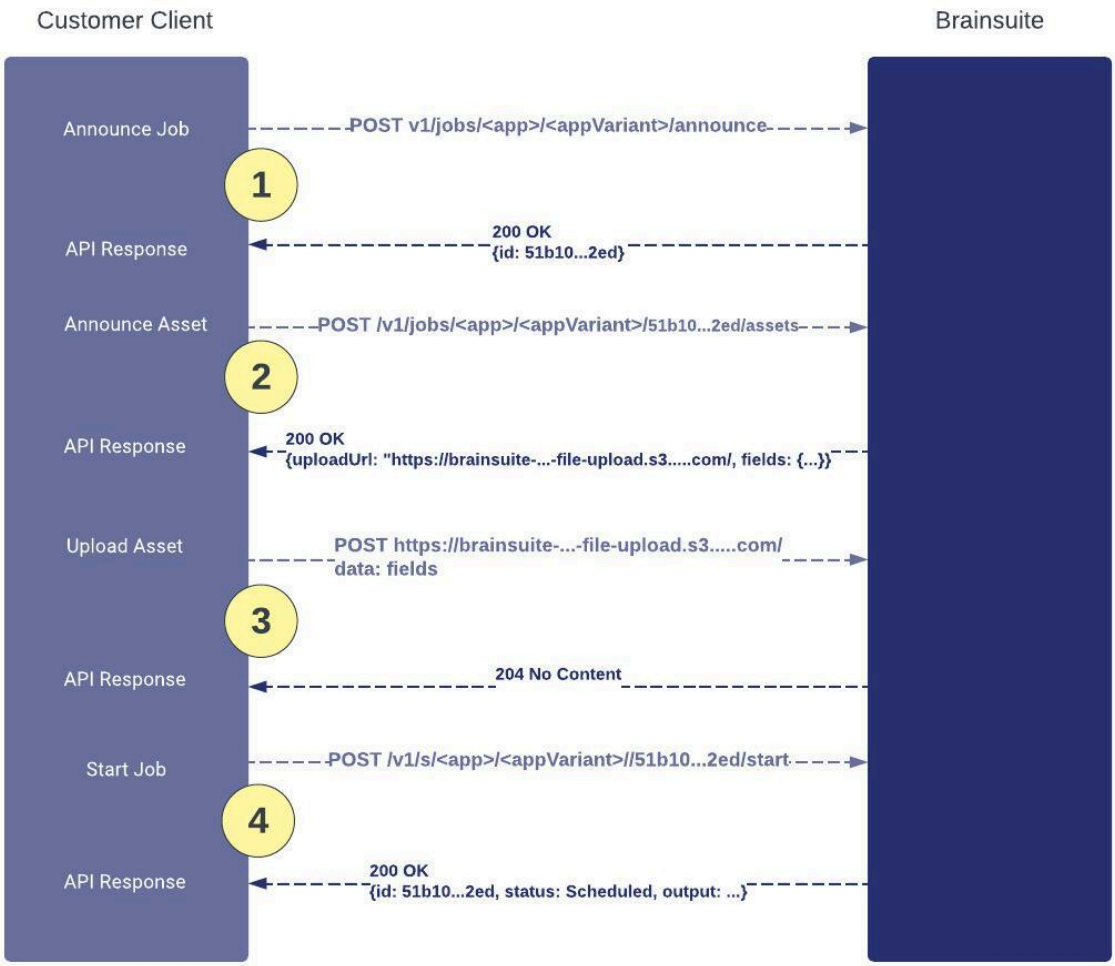
Announce-Job Workflow

Use the Announce-Job Workflow when assets are available as **files** that need to be uploaded before starting a job. This workflow follows a structured process where each asset is announced, uploaded, and assigned to the job before execution.

A detailed description of the endpoints specific to the apps are listed in the reference menu on the right or click here.

1. Customer client sends an API request to announce a job. The Brainsuite API endpoint responds with a job ID and the status `Announced`.
2. Then the customer client sends an API request to announce an asset for the job by setting the job ID as path parameter. The Brainsuite API responds with an upload URL and the associated credentials (fields) to allow the upload of an asset.
3. To upload an asset, the customer client sends an API request to the upload URL. The request contains the credentials and references the local file which shall be uploaded. The upload URL endpoint responds with an `204 No Contents` status code on a successful upload. If multiple assets are required, steps 2 and 3 must be repeated for each asset.

4. Finally, the customer client sends a request to start the job. The request contains additional input information, which are needed to create the report. The Brainsuite API responds with the job information provided and its current status.



After a successful start of the job, use the asynchronous poll-based API to check the status of the started job and to retrieve the report.

Status Information

Announced	Job is announced
Scheduled	Job is scheduled to be processed
Created	Job is processing
Succeeded	Job finished successfully
Failed	Job finished with an error
Stale	Job was not started

Upload multiple assets

Some apps allow **multiple assets** to be uploaded for a single job, such as a static image and an environment image. Additionally, some apps support **multiple legs**, where each leg represents a variation of an asset.

When using the Announce-Job Workflow, each asset must be announced before uploading and referenced correctly when starting the job.

Announce and upload assets

1. Announce an asset

Each asset must be announced before it can be uploaded. The announcement request requires a unique assetId, which is used throughout the workflow.

Example body

Copy

```
{
  "assetId": "staticImage",
  "name": "image.png"
}
```

The customer client sends an API request to announce an asset by providing the `jobId`, `assetId` and the `name` of the asset. The Brainsuite API responds with an upload URL and credentials. **Note**: Refer to the assets endpoint in the app-specific OpenAPI reference for the response structure.

2. Upload the asset

Once an asset is announced, it must be uploaded to the upload URL provided in the response. The request must include the upload credentials and the file to be uploaded.

Note: Repeat steps 1 and 2 for each asset you want to upload. Each asset must have a unique assetId.

Example cURL Command

Expand to view

Example Python Code

Expand to view

Assign Assets to Legs (if supported)

3. Start the job

Once all assets are uploaded, the customer client sends an API request to start the job. The request must include the legs and reference the correct assetId for each variation.

Example body

Note: This is an example. The actual request structure may vary depending on the app. Refer to the start endpoint in the app-specific OpenAPI reference for details.

Copy

```
{
  "legs": [
    {
      "name": "Asset Variation 1",
      "staticImage": {
        "assetId": "staticImage1"
      },
      "environmentImage": null
    },
    {
```

```
        "name": "Asset Variation 2",
        "staticImage": {
          "assetId": "staticImage2"
        },
        "environmentImage": null
      }
    ]
  }
```

Some apps may require additional assets such as environment images. These should be uploaded like any other asset and referenced in the corresponding leg.

Summary

- The **legs** array is optional and depends on the app's capabilities.
- Each leg must reference an `assetId` that corresponds to an announced and uploaded asset.
- If an app supports different asset types (e.g., `staticImage`, `environmentImage`), they should be linked accordingly.
- A job can contain a maximum of 10 legs.

After completing these steps, the job is ready to start, and the assets are assigned correctly.

Retrieve Report

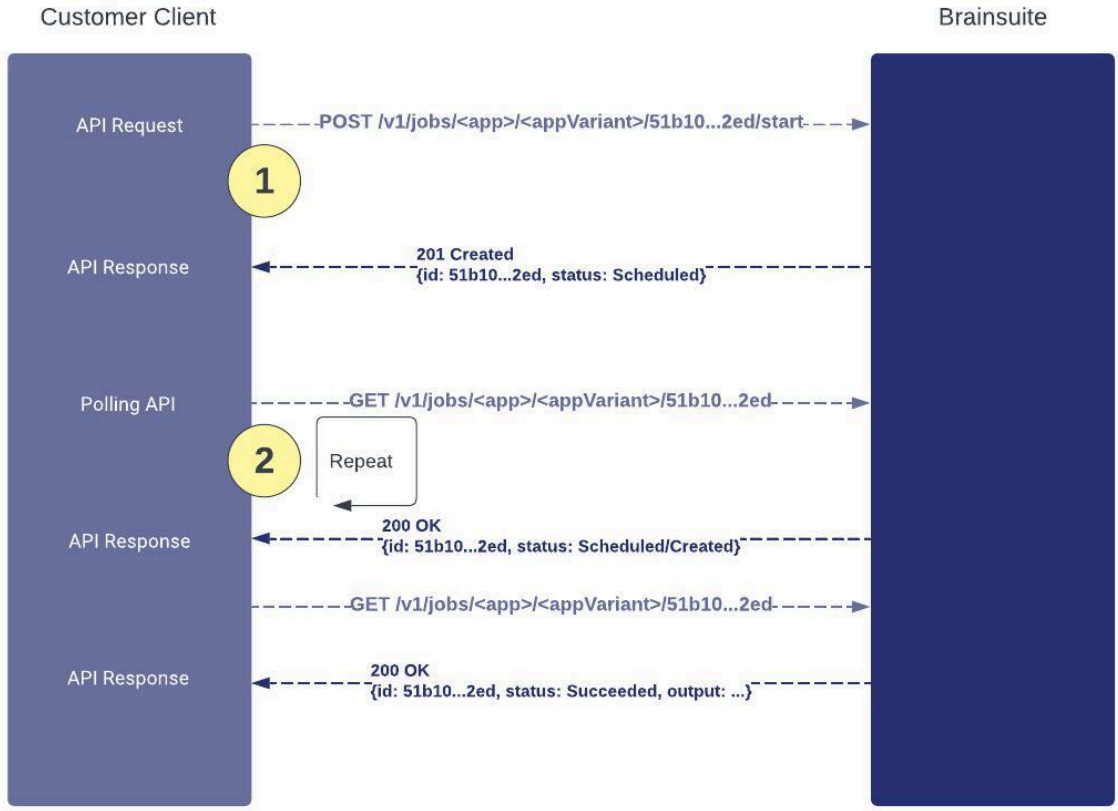
The standard Brainsuite API processes requests synchronously. For most tasks, Brainsuite sends a response shortly after the system finishes processing the call. This is not always possible with image and video processing, where times to complete are unpredictable and delayed response can lead to timeout issues. With the Brainsuite Asynchronous API, we introduce asynchronous processing of starting a job. Immediately after receiving an asynchronous API request, Brainsuite schedules the job and the system returns a `Scheduled` status along with a job ID.

You can poll later with a different API request to check the status of the original request corresponding to the job ID.

Asynchronous API Workflow

Using the asynchronous API follows this poll-based API workflow:

1. Customer client sends a request to the Brainsuite API endpoint to start a job. The asynchronous API endpoint responds with the job ID and the current status.
2. Customer client sends the job ID to the Polling API endpoint to find out the updated status of the request. The Polling API responds with the status of the asynchronous request. If the status is either `Scheduled` or `Created` the customer client continuous to send a request in a specified time interval (e.g. every 30 seconds), until the response status is not `Scheduled` or `Created` anymore.



Error handling

Sometimes things go wrong. Here are some tips to help you to handle errors in your API requests.

Use the OpenAPI "Try it out" feature of the OpenAPI reference to test your requests. If you are having trouble with the API, we recommend using a tool like Postman to get more insights of your API requests.

⌄Error Handling Steps

Report

The output key in the response of a successful job contains report information and the KPIs. The Executive Summary contains a brief overview how the asset performed. The kpis key contains detailed information for each KPI like visualizations, scores and sometimes additional data.

Important note: The visualizations expire after one hour after retrieval of a successful response.

More information about the meaning of KPIs can be found in the Academy.

Export Cockpit

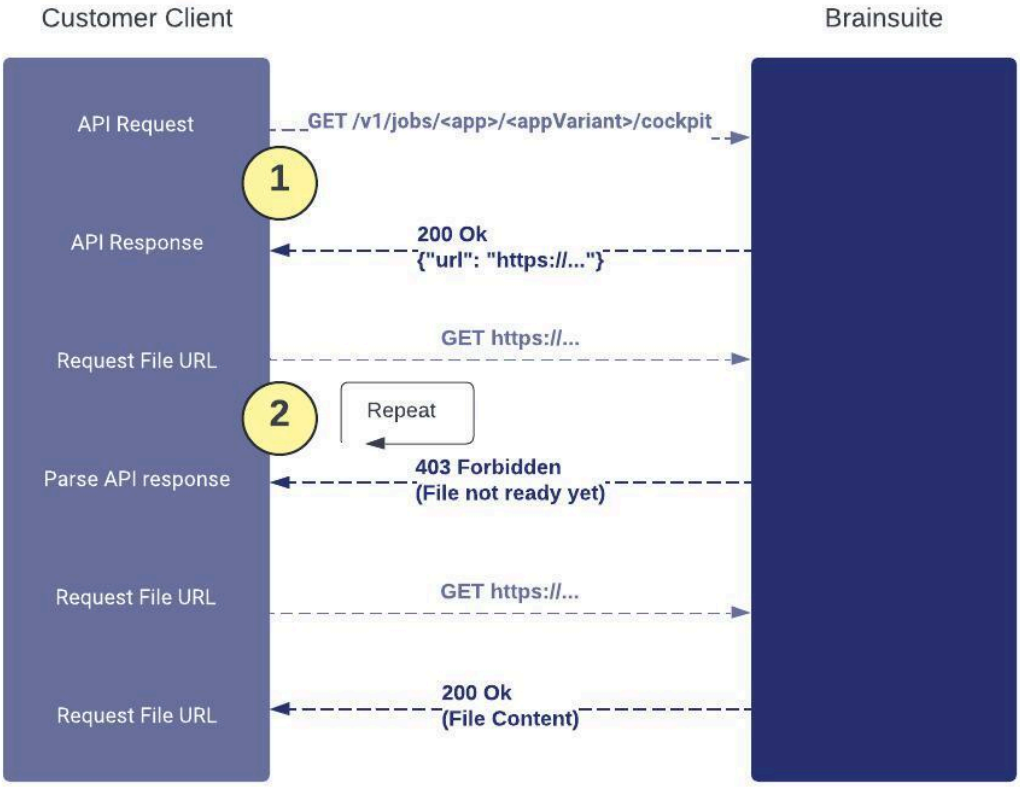
Brainsuite enables programmatic export of Cockpit data, which consists of all jobs generated by an App. The data is available in CSV format and can be exported via the Brainsuite API.
The export endpoint is documented in the OpenAPI reference.

Asynchronous Export Process

Due to the potentially large data volume, exports are processed **asynchronously**.

1. Upon initiating an export, the API returns a **file address**.
2. The **file address** can be **polled** to check the export status. If the file is not yet available, the URL responds with a `403 Forbidden` status. Once the export is complete, the request returns a `200`

OK status, allowing the file to be downloaded.



Important note: The file address expires after one hour.